PROJECT GUIDE | DATA ANALYTICS & MACHINE LEARNING

Ola Driver Churn Prediction

End-to-end explanation: business problem, data, EDA, statistics, feature engineering, modeling, evaluation, SHAP, deployment, and interview preparation

Dataset	ola_driver.csv: 19,104 monthly driver records, 14 columns, 2019-2020
Goal	Predict which drivers will leave (churn) and explain why
Tools	Python, Pandas, NumPy, Matplotlib, Seaborn, SciPy, scikit-learn, XGBoost, LightGBM, CatBoost, Optuna, SHAP, Streamlit, Pickle
Final model	Stacking ensemble (XGBoost + LightGBM + CatBoost + Random Forest -> Logistic Regression)
Headline result	AUC 0.895 | Recall 0.76 | Precision 0.35 | F1 0.48 (threshold 0.10)
Top churn drivers	Low quarterly rating, short tenure / few active months, city, low business value

How to use this guide: read Sections 1-12 in order once to understand the project end to end. Then use Section 16 (Interview Q&A) and Section 17 (Cheat Sheet) for revision right before the interview.

Contents

1. The Project in One Page . . . 3
2. Business Problem and Why It Matters . . . 4
3. The Dataset . . . 4
4. Step 1: Loading and Inspecting the Data . . . 5
5. Step 2: Feature Engineering . . . 6
6. Step 3: Exploratory Data Analysis (EDA) . . . 7
7. Step 4: Statistical Analysis . . . 7
8. Step 5: Data Preprocessing . . . 9
9. Step 6: Modeling . . . 10
10. Step 7: Evaluation and Threshold Tuning . . . 12
11. Step 8: Explainability with SHAP . . . 14
12. Step 9: Scoring, Deployment and Project Structure . . . 15
13. Business Insights and Recommendations . . . 16
14. Challenges, Limitations and Improvements . . . 17
15. SQL and KPIs for the Data Analyst Role . . . 18
16. Interview Questions and Answers . . . 19
17. Cheat Sheet: Numbers and Facts to Remember . . . 22

1. The Project in One Page

The 30-second pitch (memorize this)

> **Say this when asked "Tell me about your project"**
>
> "Ola loses drivers constantly, and replacing a driver costs much more than keeping one. I used Ola's monthly driver data, about 19,000 records, to predict which drivers are likely to leave. I cleaned the data and engineered features in Pandas, did exploratory analysis and hypothesis testing with a chi-square test and a t-test, then built classification models: first XGBoost, then a stacking ensemble tuned with Optuna. Because churners are only about 8% of records, I focused on recall and F1 rather than accuracy and tuned the decision threshold. The final model catches about **76% of churners** with an **AUC of about 0.89**. SHAP showed that **low quarterly rating, short tenure and low business value** are the strongest churn signals, and I turned that into retention recommendations and a risk-ranked driver list on a Streamlit dashboard."

The 2-minute story (structure: Problem -> Data -> Analysis -> Model -> Impact)

1. **Problem:** Driver churn hurts ride availability, rider experience and revenue. Ola wants to act *before* drivers leave.
2. **Data:** 19,104 rows, one row per driver per month, with demographics (age, gender, city, education), job details (joining designation, grade, income), performance (quarterly rating, business value) and dates (joining date, last working date).
3. **Target:** Churn = 1 when *LastWorkingDate* is present, which happens on the driver's final month.
4. **Analysis:** EDA showed churners have lower ratings, shorter tenure and lower business value. Churn ranges from 5.7% to 12.5% across cities. A chi-square test found education level is related to churn ($p = 0.0086$), and a t-test found churners have much shorter tenure ($t = -23.1$, p close to 0).
5. **Modeling:** Started with XGBoost and compared Logistic Regression and Random Forest. Showed that accuracy is misleading on imbalanced data. Built a stacking ensemble and tuned the threshold to 0.10.
6. **Result:** AUC 0.895, recall 0.76, F1 0.48.
7. **Impact:** A ranked list of at-risk drivers, explanations for each flag (SHAP), and concrete actions: coach low-rated drivers, strengthen onboarding, reward high performers, fix high-churn cities.

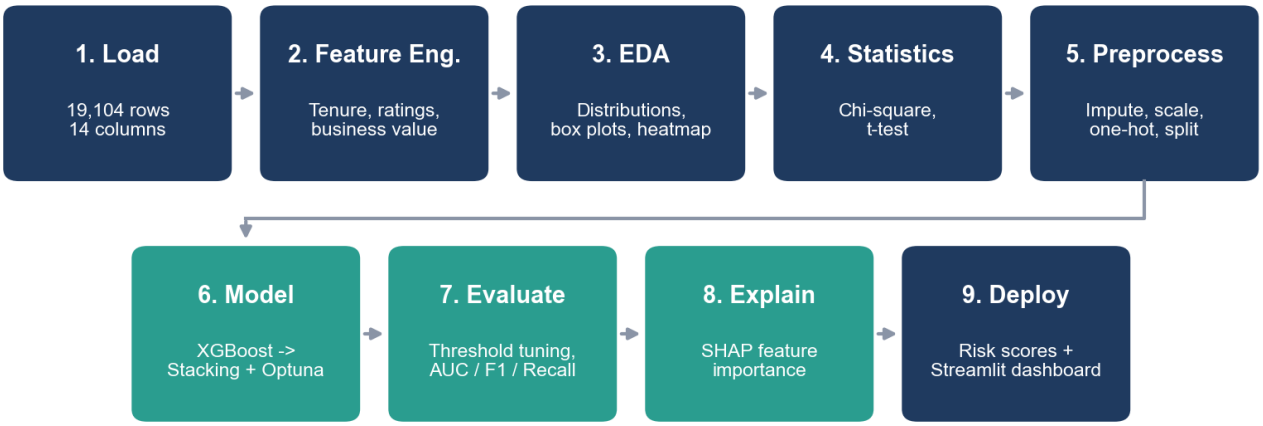


Figure 1. The end-to-end pipeline. Blue = data analysis stages, teal = machine learning stages.

2. Business Problem and Why It Matters

What is churn?

Churn (also called **attrition**) means a customer or worker stops using or working for a company. Here it means a driver stops driving for Ola. Ride-hailing platforms depend on having enough drivers: fewer drivers means longer waiting times, more cancelled rides, unhappy riders and lost revenue.

Why predict it?

- **Recruiting is expensive.** Finding, verifying and onboarding a new driver costs much more than keeping an existing one (the project documents quote 5-7x).
- **Action needs lead time.** Once a driver has left, it is too late. A prediction lets Ola step in early with bonuses, training, better routes or support.
- **Budgets are limited.** Ola cannot give every driver a bonus. A risk score focuses spending on the drivers most likely to leave.

Two questions the project answers

Question	Type of work	Output
Who is likely to leave?	Predictive (machine learning)	Churn probability and risk rank per driver
Why do drivers leave?	Descriptive / diagnostic (EDA, statistics, SHAP)	Key churn factors and business recommendations

> **Data analyst framing**
>
> For a data analyst role, stress the **why**: the EDA, the statistical tests, the segment analysis (city, rating, tenure) and the recommendations. The model is a tool; the insight and the business action are the deliverable.

3. The Dataset

The file **ola_driver.csv** has **19,104 rows and 14 columns**. The most important thing to understand is the grain (what one row represents): **each row is one driver in one month**. The same driver appears in many rows, one per month they were active, across 2019 and 2020.

Data dictionary

Column	Type	Meaning	Notes
Unnamed: 0	int	Row index saved by mistake when the CSV was written	Junk, must be dropped
MMM-YY	date	Reporting month of the record	Month-first format, e.g. 01/01/19 = Jan 2019
Driver_ID	int	Unique driver identifier	Used for grouping, not as a feature
Age	float	Driver age	Has some missing values
Gender	0/1	0 = male, 1 = female	Has some missing values
City	category	City code, C1 to C29	29 cities; one-hot encoded
Education_Level	0/1/2	10th / 12th / graduate	Ordinal
Income	int	Monthly average income of the driver	Right-skewed

Column	Type	Meaning	Notes
Dateofjoining	date	Date the driver joined Ola	Day-first, e.g. 24/12/18
LastWorkingDate	date	Date the driver left	**Blank = still working.** Source of the target
Joining Designation	1-5	Designation (level) at joining	
Grade	1-5	Grade (level) at the time of reporting	
Total Business Value	int	Business (revenue) the driver brought in that month	Can be negative (refunds, cancellations)
Quarterly Rating	1-5	Performance rating for the quarter	Higher is better

How the target variable (Churn) is defined

```
df['Churn'] = df['LastWorkingDate'].notnull().astype(int)
```

LastWorkingDate is filled in only on a churned driver's **final month row**. All their earlier rows, and every row of a driver who never left, have it blank. So Churn = 1 means *"this month was the driver's last month"*.

Level	Churn rate	What it means
Row level (used in this project)	**8.46%**	8.46% of driver-months are a driver's final month
Driver level	Much higher (around two-thirds in the public version of this dataset)	Share of unique drivers who left at any point; check with a groupby on Driver_ID

Interview trap: "Only 8% churn? I thought most Ola drivers churned."

Answer: "8.46% is the row-level rate: the share of driver-months that were a driver's last month. At the driver level, a much larger share of drivers eventually leave. The model is predicting whether a given month will be a driver's last, which is why the classes are imbalanced." This shows you understand the grain of your own data.

Missing values

- **LastWorkingDate** is blank for most rows, **by design** (the driver had not left that month). It is not "missing data"; it is information, and it becomes the target.
- **Age** and **Gender** have a small number of missing values. These were imputed (median for numbers, mode for categories).

4. Step 1: Loading and Inspecting the Data

```
import pandas as pd
df = pd.read_csv('ola_driver.csv')

df.head()            # first 5 rows: see what the data looks like
df.info()            # column types and non-null counts
df.describe()        # min, max, mean, quartiles of numeric columns
df.isnull().sum()    # missing values per column
```

Command	What you learn from it
df.shape	19,104 rows x 14 columns
df.info()	Dates are stored as text (object) and must be converted; Age/Gender have nulls

Command	What you learn from it
df.describe()	Business Value can be negative; Income is skewed; ratings are 1-4 in practice
df.isnull().sum()	Most nulls are in LastWorkingDate (expected), a few in Age and Gender
df['Driver_ID'].nunique()	How many unique drivers there are (the grain check)

5. Step 2: Feature Engineering

Feature engineering means creating new, more useful columns from raw data. Raw columns describe one month; engineered features describe the driver's history and behavior, which predicts churn much better.

Features created

Feature	How it is computed	Why it helps predict churn
Tenure	Months between Dateofjoining and the reporting month (MMM-YY)	New drivers leave more often
avg_quarterly_rating	Mean Quarterly Rating per driver (groupby)	Overall performance level
total_business_value	Sum of Total Business Value per driver	Lifetime value to Ola
Monthly_Business_Value	total_business_value / Tenure	Productivity per month
active_months	Number of unique months a driver appears	Engagement / loyalty
avg_monthly_business_value	Mean of the driver's monthly business value	Consistency of earnings
last_active_month	Latest month the driver appears	Recency
time_since_last_active_months	End of data period minus last_active_month	Inactivity signal

Key code pattern: groupby, aggregate, merge back

```
# 1. Convert text to dates
df['MMM-YY'] = pd.to_datetime(df['MMM-YY'], errors='coerce')
df['Dateofjoining'] = pd.to_datetime(df['Dateofjoining'], errors='coerce')

# 2. Tenure in months
df['Tenure'] = ((df['MMM-YY'].dt.year - df['Dateofjoining'].dt.year) * 12 +
                (df['MMM-YY'].dt.month - df['Dateofjoining'].dt.month)).clip(lower=0)

# 3. Driver-level summaries
driver_perf = df.groupby('Driver_ID').agg(
    avg_quarterly_rating=('Quarterly Rating', 'mean'),
    total_business_value=('Total Business Value', 'sum'),
    active_months=('MMM-YY', 'nunique'),
).reset_index()

# 4. Attach the summaries back to every row of that driver
df = df.merge(driver_perf, on='Driver_ID', how='left')
```

Explain it simply

groupby('Driver_ID') collapses many monthly rows into one summary row per driver. merge(..., how='left') copies that summary back onto every monthly row, so each row knows both "what happened this month" and "what this driver is like overall". In SQL this is a GROUP BY subquery joined back with a LEFT JOIN, or a window function such as AVG(...) OVER (PARTITION BY Driver_ID).

6. Step 3: Exploratory Data Analysis (EDA)

EDA means looking at the data with summaries and charts before modeling, to understand distributions, spot problems and form hypotheses about what drives churn.

Chart	Purpose	What it showed
Histograms (numeric columns)	Shape of each distribution	Income and Business Value are right-skewed with outliers; many drivers have low tenure
Count plots (Gender, City, Education)	How many records per category	Education is balanced; City has 29 levels with different sizes
Box plots (feature vs Churn)	Compare churners and non-churners	Churners have **lower Quarterly Rating**, **shorter Tenure** and **lower Business Value**
Stacked bars (category vs Churn)	Churn share inside each category	Differences across cities; small differences by gender and education
Correlation heatmap	Linear relationships between numeric columns	Rating and Tenure negatively correlated with churn; Grade and Joining Designation correlated with each other and with Income

Key EDA findings

1. **Performance:** drivers with a low quarterly rating (1-2) churn much more.
2. **Tenure:** newer drivers churn more; the first months are the riskiest.
3. **Business value:** low-earning drivers churn more. If driving doesn't pay, drivers leave.
4. **Geography:** churn varies about 2x across cities.
5. **Demographics:** gender and education show only small differences.

> **Reading a box plot**
>
> The box covers the middle 50% of values (Q1 to Q3), the line inside is the median, the whiskers extend to 1.5 x IQR, and dots beyond are outliers. If the churn box sits clearly lower than the non-churn box, that feature separates the two groups.

7. Step 4: Statistical Analysis

Overall and segmented churn rates

Overall row-level churn rate: **8.46%**. Segment rates (churn rate within each group):

Segment	Churn rate	Takeaway
Gender 0 / 1 / Unknown	8.52% / 8.36% / 9.62%	Practically no difference
Education 0 / 1 / 2	9.17% / 7.68% / 8.65%	Small but statistically significant difference
City (29 cities)	5.67% (C29) to 12.5% (C17)	About 2x spread; location matters

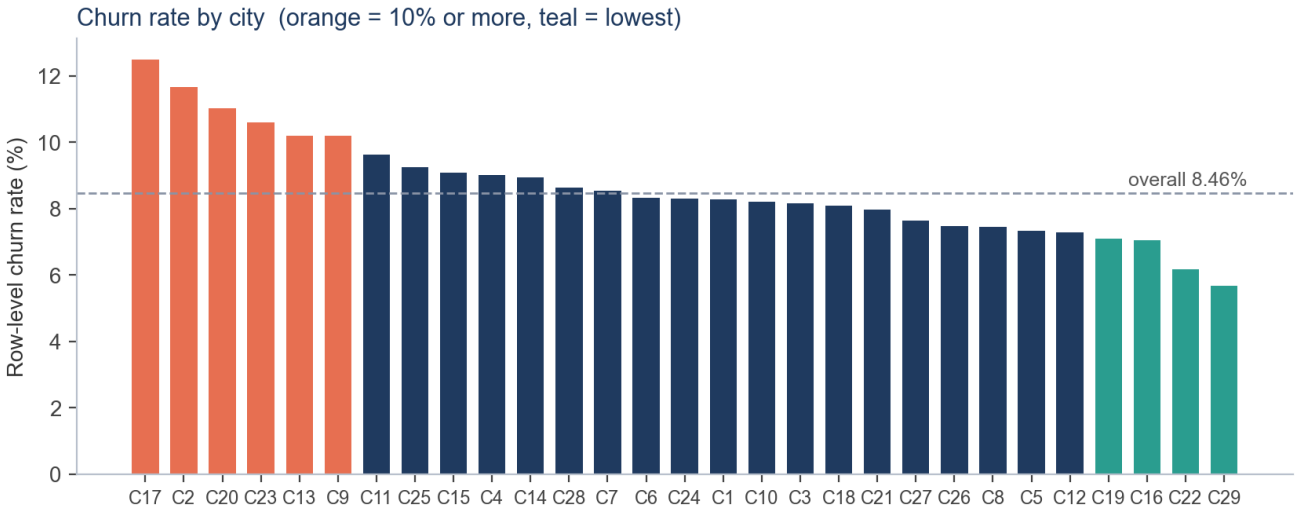


Figure 2. Churn rate by city. C17, C2, C20, C23, C13 and C9 are at 10% or above; C29 and C22 are lowest.

Chi-square test of independence: Education_Level vs Churn

Item	Value
Used for	Two **categorical** variables: are they related or independent?
H0 (null hypothesis)	Education level and churn are independent (unrelated)
H1 (alternative)	Education level and churn are related
Result	Chi-square = 9.51, degrees of freedom = 2, **p = 0.0086**
Decision	p < 0.05, so reject H0: education level is significantly associated with churn

```
from scipy.stats import chi2_contingency
table = pd.crosstab(df['Education_Level'], df['Churn'])
chi2, p, dof, expected = chi2_contingency(table)
```

How it works: it compares the *observed* counts in the crosstab with the counts *expected* if the variables were unrelated. A big gap means a large chi-square and a small p-value. Degrees of freedom = (rows - 1) x (columns - 1) = (3 - 1) x (2 - 1) = 2.

Welch's t-test: Tenure of churners vs non-churners

Item	Value
Used for	Compare the **means** of a numeric variable between two groups
H0	Mean tenure is the same for churners and non-churners
Result	**t = -23.13, p close to 0**
Decision	Reject H0. The negative t means churners have significantly **shorter** tenure
Why Welch?	equal_var=False: the groups have very different sizes and variances, so the standard (Student's) t-test assumption of equal variances does not hold

```
from scipy.stats import ttest_ind
t, p = ttest_ind(df[df.Churn == 1]['Tenure'], df[df.Churn == 0]['Tenure'], equal_var=False)
```

Statistical vs practical significance (great point to raise)

With 19,000 rows, even tiny differences become "statistically significant". Education is significant (p = 0.0086), but the gap is only about 1.5 percentage points (7.7% vs 9.2%), which is a small effect. Tenure and rating have both significance **and** large effects, so those deserve business action.
Rule: a p-value tells you whether an effect is likely real; the effect size tells you whether it matters.

Choosing the right test

Situation	Test
Categorical vs categorical (e.g. Education vs Churn)	Chi-square test of independence
Numeric vs 2 groups (e.g. Tenure: churn vs no churn)	t-test (Welch if variances differ); Mann-Whitney U if not normal
Numeric vs 3+ groups (e.g. Income across cities)	ANOVA; Kruskal-Wallis if not normal
Numeric vs numeric (e.g. Income vs Business Value)	Pearson correlation (Spearman for ranks / non-linear)
Two proportions (e.g. churn rate city A vs city B)	Two-proportion z-test

8. Step 5: Data Preprocessing

Machine learning models need clean, numeric input. Preprocessing turns the engineered table into a numeric matrix.

1. **Impute missing values:** numeric columns get the **median** (robust to outliers and skew); categorical columns get the **mode** (most frequent value).
2. **Drop non-features:** Driver_ID (an identifier, not a behavior), the raw date columns, the index column Unnamed: 0, and Churn itself (the target).
3. **Scale numeric features with StandardScaler:** z = (x - mean) / standard deviation, so every feature has mean 0 and standard deviation 1. Logistic Regression needs this; tree models don't, but it keeps the pipeline consistent.
4. **One-hot encode categorical features (City, Gender):** each city becomes its own 0/1 column. Label encoding (C1 = 1, C2 = 2...) would invent a fake order. handle_unknown='ignore' stops the model crashing on a new city.
5. **ColumnTransformer:** applies the scaler to numeric columns and the encoder to categorical columns in one reusable object, which is saved to disk (preprocessor.pkl) for the dashboard.
6. **Train/test split 80/20, stratified:** 15,283 training rows and 3,821 test rows. stratify=y keeps the 8.46% churn ratio the same in both sets.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numerical_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
X_train = preprocessor.fit_transform(X_train)   # learn mean/std/categories from TRAIN only
X_test  = preprocessor.transform(X_test)        # apply the same transformation to TEST
```

After one-hot encoding, the column count grows from about 14-19 raw columns to **43** (final pipeline) or **48** (notebook version, which still included Unnamed: 0 and a few extra columns).

Why fit only on the training set?

If the scaler learns its mean and standard deviation from the full data, information from the test set leaks into training and the test score looks better than reality. The correct order is: split first, fit on train, transform test.

9. Step 6: Modeling

9.1 Class imbalance: the central challenge

Only about 8.5% of rows are churn = 1. A model can be lazy and predict "no churn" for everyone and still look accurate. Techniques used to fight imbalance:

- **Class weighting:** XGBoost's *scale_pos_weight* = (number of non-churn) / (number of churn), about 10.8. Each churner counts about 11 times as much during training.
- **Stratified split** so both sets have the same churn ratio.
- **Right metrics:** recall, precision, F1 and AUC instead of accuracy.
- **Threshold tuning:** lower the probability cut-off from 0.5 to 0.10.
- Alternatives worth knowing: SMOTE (synthetic oversampling), random undersampling.

9.2 Baseline: XGBoost with class weighting

```
from xgboost import XGBClassifier
xgb = XGBClassifier(n_estimators=300, learning_rate=0.05, max_depth=6,
                    subsample=0.8, colsample_bytree=0.8,
                    scale_pos_weight=neg / pos,   # ~10.8, handles imbalance
                    eval_metric='logloss', tree_method='hist', random_state=42)
xgb.fit(X_train, y_train)
```

Accuracy	Precision	Recall	F1	AUC
0.817	0.269	**0.678**	0.385	0.858

9.3 Hyperparameter tuning with RandomizedSearchCV

Searched 20 random combinations of n_estimators, learning_rate, max_depth, subsample, colsample_bytree, gamma and min_child_weight with 3-fold cross-validation, scoring on F1. Best: max_depth = 3, learning_rate = 0.1, n_estimators = 300, subsample = 0.7, colsample_bytree = 0.6, gamma = 1.

AUC improved to **0.882**, but recall **fell to 0.20**. The tuned model was built without *scale_pos_weight*, so it stopped prioritizing churners. **Lesson:** a better AUC does not mean a better business model; the imbalance handling must be kept during tuning.

9.4 Model comparison and the accuracy paradox

Model	Accuracy	Precision	Recall	F1	AUC
Logistic Regression	0.916	0.600	0.009	0.018	0.856
Random Forest	0.915	0.500	0.028	0.053	0.872
XGBoost (tuned)	0.917	0.529	0.195	0.285	0.882

Model	Accuracy	Precision	Recall	F1	AUC
XGBoost (baseline, weighted)	0.817	0.269	0.678	0.385	0.858
Stacking ensemble, threshold 0.10	**0.858**	**0.346**	**0.762**	**0.476**	**0.895**

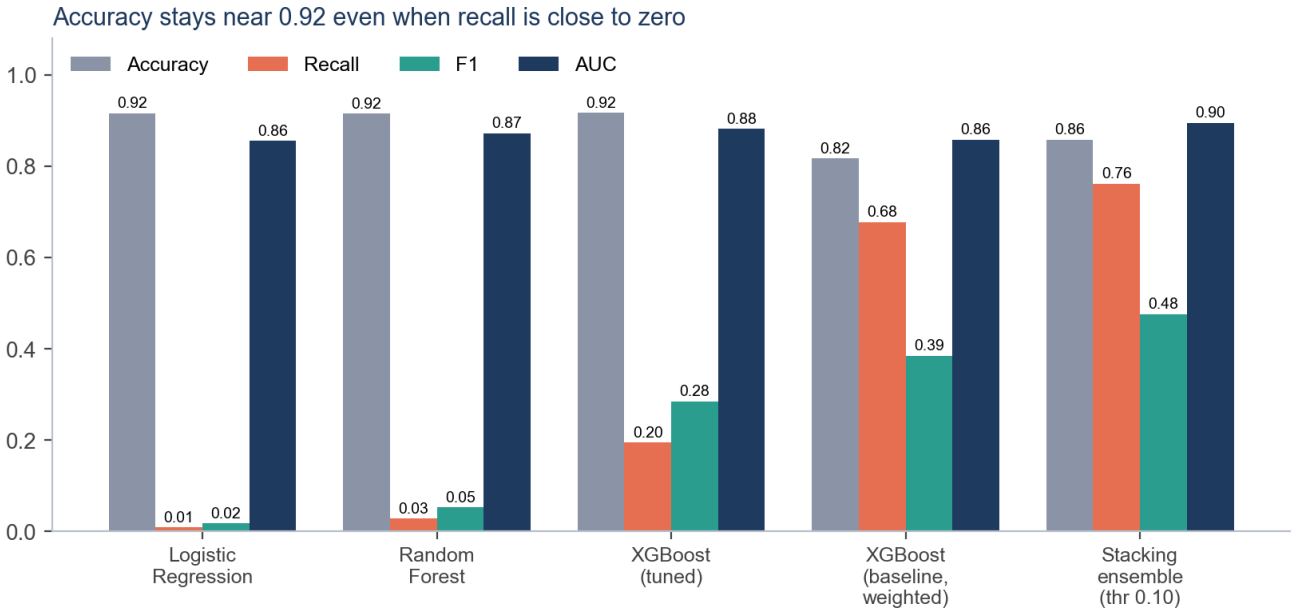


Figure 3. Accuracy hides failure: Logistic Regression scores 92% accuracy while catching under 1% of churners.

The accuracy paradox (know this cold)

A model that always predicts "no churn" gets 3,498 / 3,821 = **91.5% accuracy** on the test set and catches **zero** churners. Logistic Regression and Random Forest are barely better than this useless baseline. On imbalanced data, judge models by recall, precision, F1 and AUC.

9.5 Final model: stacking ensemble

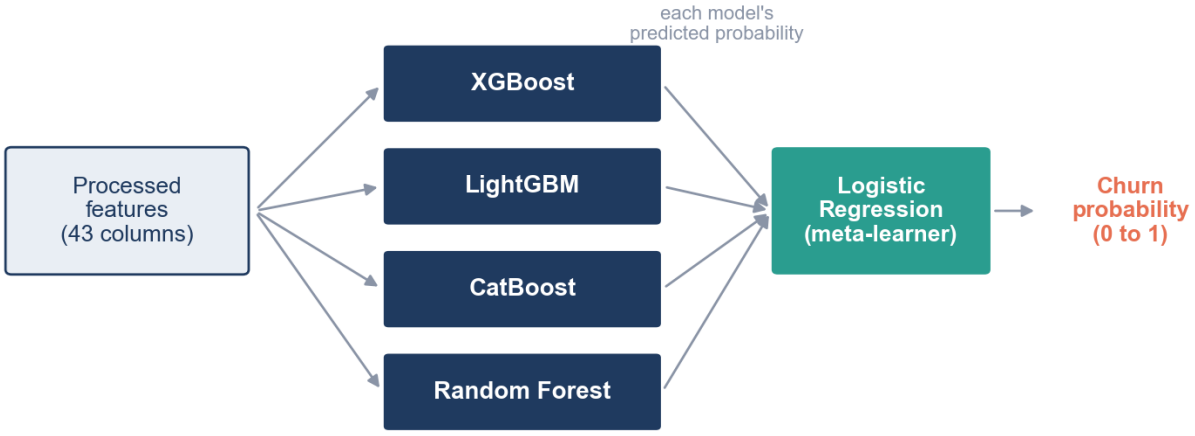


Figure 4. Stacking: four different base models feed their predictions into a Logistic Regression meta-learner.

Stacking trains several different models (base learners) and then a simple model (the meta-learner) that learns how much to trust each one. Each model makes different kinds of mistakes, so combining them gives more robust predictions. The base learners' predictions for training are produced with cross-validation (out-of-fold) so the meta-learner does not learn from predictions the base models made on their own training data.

Base learner	What it is	Strength
XGBoost	Gradient boosting with regularization	Strong on tabular data and feature interactions
LightGBM	Gradient boosting that grows trees leaf-wise using histograms	Very fast, handles many features
CatBoost	Gradient boosting with built-in categorical handling	Good with categories, less overfitting
Random Forest	Bagging: many independent deep trees, averaged	Stable, low variance, adds diversity
Logistic Regression (meta)	Linear model on the 4 predicted probabilities	Simple, calibrated combination

Hyperparameters were tuned with **Optuna** (20 trials, optimizing F1). Optuna uses Bayesian optimization (the TPE algorithm): it learns from earlier trials which regions of the parameter space look promising, so it usually finds good settings faster than grid or random search.

9.6 Bagging vs boosting vs stacking

Method	How it works	Example	Mainly reduces
Bagging	Train many models in parallel on bootstrap samples; average or vote	Random Forest	Variance (overfitting)
Boosting	Train models one after another; each fixes the previous one's errors	XGBoost, LightGBM, CatBoost	Bias (underfitting)
Stacking	A meta-model learns how to combine different model types	This project's ensemble	Both, via diversity

10. Step 7: Evaluation and Threshold Tuning

The confusion matrix

A classifier outputs a probability. A **threshold** turns it into a yes/no decision (probability >= threshold means "churn"). Comparing predictions with reality gives four outcomes:

	Predicted: Stay	Predicted: Churn
Actual: Stay	True Negative (TN): correctly left alone	False Positive (FP): false alarm, wasted retention offer
Actual: Churn	False Negative (FN): missed churner, driver lost	True Positive (TP): churner caught in time

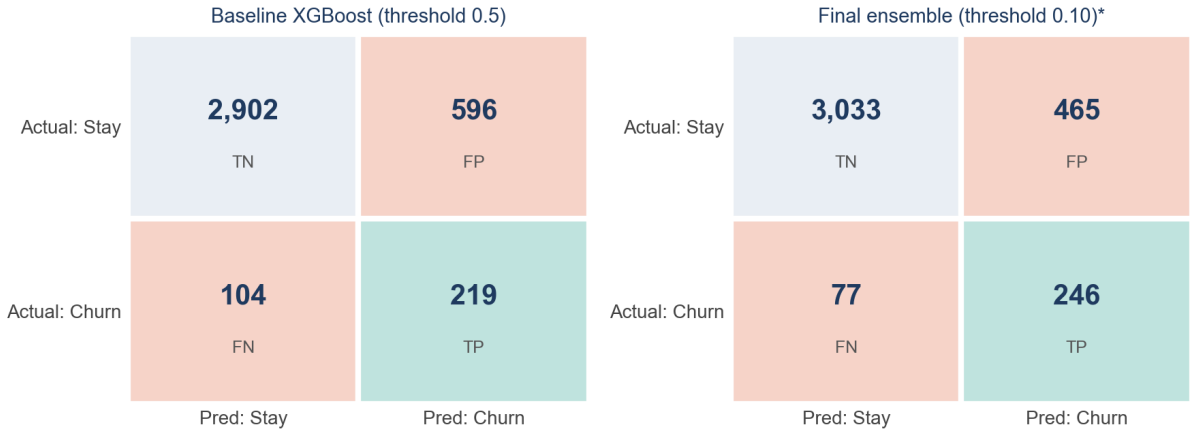


Figure 5. Confusion matrices on the 3,821-row test set (323 actual churners).

Correction to the project README

The README and submission notebook show a confusion matrix with **0 false positives**. That cannot be right: 0 false positives would mean 100% precision, but the reported precision is 34.6%. Working back from the reported metrics (323 churners, recall 0.762, precision 0.346, accuracy 0.858) gives the matrix above: TP about 246, FN about 77, FP about 465, TN about 3,033.
Check: recall = 246 / 323 = 0.76; precision = 246 / 711 = 0.35; accuracy = (246 + 3,033) / 3,821 = 0.858. Use these numbers in the interview.

Metrics in plain English

Metric	Formula	Plain-English question	Final value
Accuracy	(TP + TN) / all	What share of all predictions were right?	0.858
Precision	TP / (TP + FP)	Of drivers we flagged, how many actually left?	0.346
Recall (sensitivity)	TP / (TP + FN)	Of drivers who left, how many did we catch?	0.762
F1 score	2PR / (P + R)	Harmonic mean: balance of precision and recall	0.476
Specificity	TN / (TN + FP)	Of drivers who stayed, how many did we leave alone?	~0.867
ROC-AUC	Area under ROC curve	Chance a random churner is scored higher than a random stayer	0.895

- **ROC curve:** plots the true positive rate (recall) against the false positive rate at every possible threshold. AUC = 0.5 is a coin flip, 1.0 is perfect. AUC does not depend on the threshold, so it measures how well the model *ranks* drivers.
- **Precision-recall curve:** often more informative than ROC when positives are rare. Average precision summarizes it.

Threshold tuning: why 0.10?

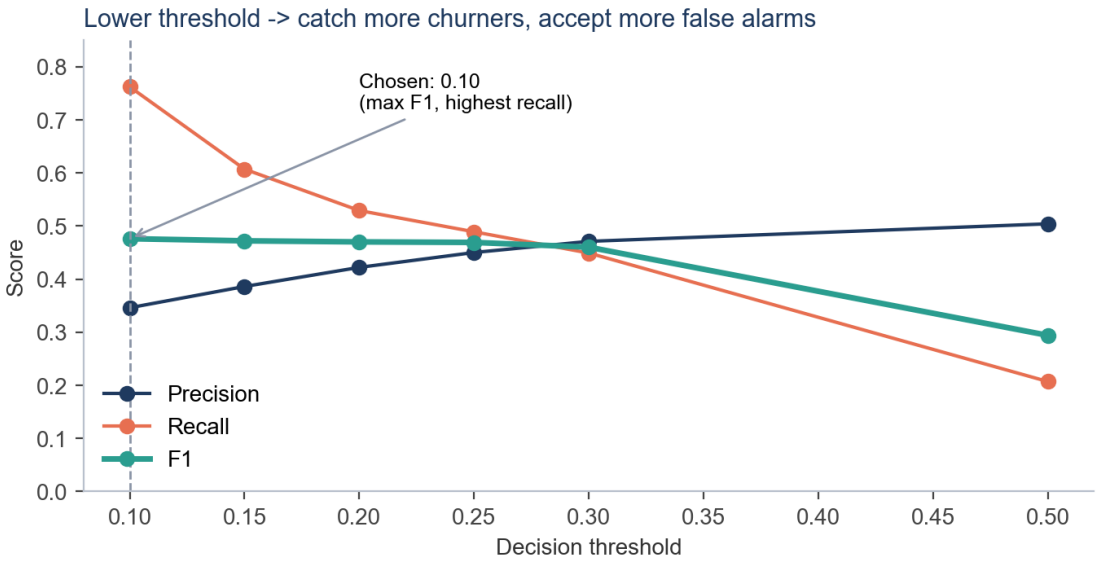


Figure 6. Precision, recall and F1 at different thresholds for the final ensemble.

Threshold	Precision	Recall	F1	Interpretation
0.10	**0.346**	**0.762**	**0.476**	**Selected: maximum F1 and highest recall**
0.15	0.386	0.607	0.472	More conservative

Threshold	Precision	Recall	F1	Interpretation
0.20	0.422	0.529	0.470	Higher precision
0.25	0.450	0.489	0.469	Roughly balanced
0.30	0.471	0.449	0.460	Prefers precision
0.50 (default)	0.504	0.207	0.294	Misses 4 out of 5 churners

The default 0.5 cut-off is wrong for rare events: because churn is rare, the model's probabilities rarely exceed 0.5. Lowering the threshold trades precision for recall. That is the right trade here because **a missed churner (FN) costs more than a false alarm (FP)**: losing a driver means re-recruiting and onboarding, while a false alarm costs a phone call or a small incentive.

Business translation

At threshold 0.10 the model flags about 711 test-set rows; about 246 of them really churn. Random targeting of 711 rows would find about 711 x 8.46% = 60 churners, so the model is roughly **4x more efficient than random outreach**.

11. Step 8: Explainability with SHAP

SHAP (SHapley Additive exPlanations) explains a model's predictions. It is based on Shapley values from cooperative game theory: each feature is a "player", and SHAP fairly divides the prediction (the "payout") among the features. For every driver it says how much each feature pushed the churn risk up or down from the average.

```
import shap
explainer = shap.TreeExplainer(xgb_model)     # fast, exact method for tree models
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)        # global importance + direction
```

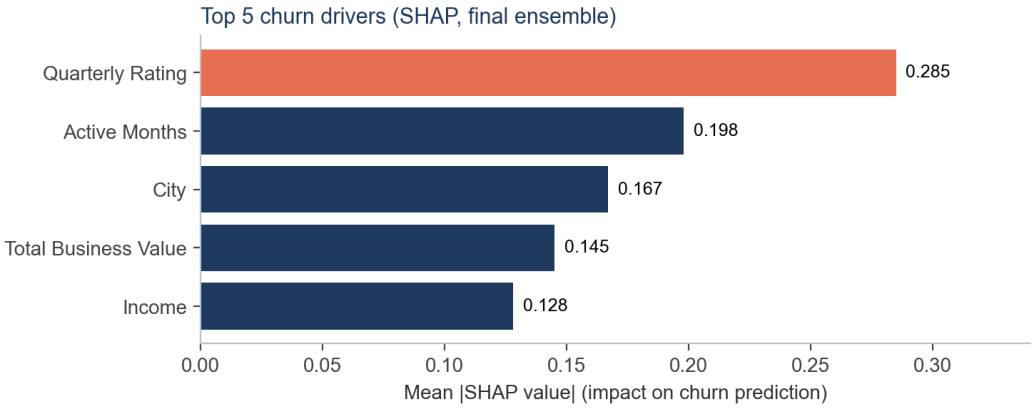


Figure 7. Top five features by mean absolute SHAP value (as reported for the final ensemble).

Reading a SHAP beeswarm / summary plot

- Each **row** is a feature, sorted by importance (most important at the top).
- Each **dot** is one driver-month. Horizontal position = SHAP value: right pushes toward churn, left pushes away.
- **Color** = the feature's value (red = high, blue = low). Example: blue (low) rating dots on the right means low ratings increase churn risk.

What SHAP found

Feature	Direction of effect
Quarterly Rating	Low rating (1-2) strongly increases churn risk. Strongest factor.
Tenure / Active Months	Short tenure and few active months increase risk.
City	Some cities have consistently higher risk.
Total / Monthly Business Value	Low business value increases risk.
Income	Lower income is associated with higher risk.
Time since last active	Longer inactivity increases risk.
Gender, Education	Small influence compared with performance and engagement.

Why explainability matters

Operations managers will not act on a black box. SHAP lets you say "This driver is high-risk mainly because of a rating of 1 and only 2 months of tenure". That turns a score into a specific action, and it builds trust that the model learned sensible patterns. A practical fix from the project: computing SHAP was taking over 30 minutes, so the sample size and background data were reduced, cutting it to 2-3 minutes.

12. Step 9: Scoring, Deployment and Project Structure

Scoring every driver

```
df['churn_probability'] = model.predict_proba(X_processed)[:, 1]
df['churn_prediction']  = (df['churn_probability'] >= 0.10).astype(int)
df['churn_risk_rank']   = df['churn_probability'].rank(method='dense', ascending=False)
df.to_csv('data/churn_scores.csv', index=False)
```

The output is a risk-ranked list. In the notebook the top-risk drivers were IDs 897, 2632, 172, 2563 and 965 with churn probabilities of 0.77 to 0.90. The retention team would work down this list.

The pipeline orchestrator: main.py

Step	Function / file	Output
1. Load	load_data() (src/data_loading_01.py)	Raw DataFrame
2. Engineer	engineer_features() (src/feature_engineering_02.py)	Feature-rich DataFrame
3. Preprocess	preprocess() (src/preprocessing_06.py)	X_train, X_test, y_train, y_test, preprocessor
4. Train	build_ensemble(optimize=True, metric='f1', n_trials=20)	Stacking model tuned with Optuna
5. Save	pickle.dump(...)	models/preprocessor.pkl, models/training_features.pkl
6. Evaluate	evaluate() (src/evaluation_08.py)	Metrics and best threshold
7. Score	score() (src/scoring_10.py)	data/churn_scores.csv

Streamlit dashboard

- Dataset overview with filters; churn distribution charts.

- SHAP feature importance.
- Real-time prediction: enter one driver's details and get a churn probability.
- Top 20 high-risk drivers table; a debug toggle for model internals.
- Run with: `streamlit run dashboard/dashboard.py` (opens at localhost:8501).

Files in the project folder

File	What it is
Ola_Ensembling.ipynb	The main analysis notebook: EDA, statistics, XGBoost, tuning, model comparison, SHAP, recommendations. **This is where the actual outputs live.**
main.py	End-to-end pipeline script (load, engineer, preprocess, train stacking, evaluate, score)
PROJECT_SUBMISSION.ipynb	Write-up notebook: objectives, methodology, challenges, results, future work
debug_*.py	Scripts used to fix the 43-vs-44 feature mismatch between saved preprocessor and model
README.md, INDEX.md, SUBMISSION_SUMMARY.md, DELIVERABLES.md	Documentation
requirements.txt	Library versions (pandas 2.3, scikit-learn 1.7, xgboost 3.1, lightgbm 4.6, catboost 1.2, optuna 4.6, shap 0.50, streamlit 1.52)

Note on this copy of the project

The src/, data/, models/ and dashboard/ folders that main.py imports are not in this downloaded copy, so main.py cannot run as-is. The notebook outputs are the evidence for the results.

13. Business Insights and Recommendations

Finding (evidence)	Recommendation	How to measure success
Low quarterly rating means high churn (top SHAP feature, box plots)	Coaching and training for drivers rated 1-2: customer handling, route tips, mentoring	Rating improvement and churn rate of coached vs non-coached drivers
Short tenure means high churn (t = -23.1)	Structured onboarding, a buddy/mentor, incentives in the first 1-3 months, early feedback	3-month and 6-month retention of new cohorts
Low business value means high churn	Tiered rewards, guaranteed-earnings periods for new drivers, lower commission for top performers	Business value per driver; churn by earnings band
City churn ranges from 5.7% to 12.5%	Investigate C17, C2, C20, C23: competition, demand, support hubs; local incentive programs	City-level churn trend month over month
Inactivity rises before churn	Automatic re-engagement nudges when a driver's activity drops	Reactivation rate after a nudge
Gender and education matter little	Don't target retention spend by demographics; target by performance and engagement	-

Closing the loop: A/B testing

To prove a retention offer works, randomly split high-risk drivers into a treatment group (gets the offer) and a control group (doesn't). Compare churn rates with a two-proportion z-test. Without a control group you can't tell whether the offer caused the retention.

14. Challenges, Limitations and Improvements

Challenges solved during the project

Challenge	Root cause	Fix
Feature shape mismatch (model expected 43 features, got 44)	Unnamed: 0 was not dropped in preprocessing, and main.py did not save the new preprocessor, so the dashboard loaded an old one	Dropped Unnamed: 0, saved preprocessor.pkl in main.py, regenerated all artifacts
Couldn't access the one-hot encoder inside the pipeline	Nested Pipeline / ColumnTransformer objects	Navigate with named_transformers_['cat'] / named_steps['ohe']
SHAP took 30+ minutes	Too many samples for a stacked model	Smaller sample and background set (2-3 min)
Class imbalance (~8.5% churn)	Rare positive class	Class weights, F1/recall focus, threshold 0.10
Inconsistent date formats	Mixed day-first and month-first dates	Explicit parsing with errors='coerce'

Honest limitations (raise these yourself if probed)

Naming these unprompted shows maturity. Interviewers trust candidates who know where their work is weak.

1. **Data leakage from future information.** Features such as *active_months*, *time_since_last_active_months* and *total_business_value* are computed over a driver's entire history, including months after the one being predicted. A churned driver's final row "knows" the driver's activity stopped. Fix: build features only from data available up to that month (rolling / expanding windows).
2. **Same driver in train and test.** The split was by row, so one driver's months can be in both sets and the model partly memorizes drivers. Fix: GroupShuffleSplit / GroupKFold on Driver_ID, or a time-based split (train on earlier months, test on later months).
3. **Index column used as a feature in the notebook.** Unnamed: 0 (row order) was a feature in the notebook version; it was removed in the final pipeline.
4. **Scaler fitted before the split in the notebook.** Minor leakage; fixed by fitting on training data only.
5. **Date parsing.** MMM-YY is month-first (01/01/19, 02/01/19, 03/01/19 = Jan, Feb, Mar 2019), but the notebook parsed it with format '%d/%m/%y', reading it as 1, 2, 3 January. That distorts Tenure and the month-based features. Fix: format='%m/%d/%y'.
6. **Inconsistent numbers across documents.** One notebook summary mentions a "tuned LightGBM, AUC 0.9357" not shown in the code, and the README's confusion matrix is wrong. Quote the numbers in this guide.

> **How to say it in the interview**
> "Looking back, the biggest improvement would be preventing leakage: splitting by driver or by time and building features only from past data. I'd expect lower but more honest metrics, and that's the version I'd trust in production. I'd also reframe the problem at the driver level, for example predicting churn within the next 3 months, which is more actionable for the business."

Future enhancements

- Driver-level or "churn in the next N months" target with time-based validation.
- More features: rating trend (did the rating drop?), month-over-month business value change, grade promotions (Grade vs Joining Designation), income change.
- Probability calibration; a cost-based threshold (choose the threshold that maximizes expected savings).
- REST API (FastAPI) for real-time scoring; Docker; scheduled monthly retraining; drift monitoring.
- A/B testing of retention offers; Power BI / Tableau dashboard for operations teams.

15. SQL and KPIs for the Data Analyst Role

Data analyst interviews often ask you to reproduce analysis in SQL. Assume a table `ola_driver` with a proper `reporting_month` date column.

Churn rate by city (driver level)

```
SELECT City,
       COUNT(DISTINCT Driver_ID) AS drivers,
       COUNT(DISTINCT CASE WHEN LastWorkingDate IS NOT NULL THEN Driver_ID END) AS churned,
       ROUND(100.0 * COUNT(DISTINCT CASE WHEN LastWorkingDate IS NOT NULL THEN Driver_ID END)
             / COUNT(DISTINCT Driver_ID), 2) AS churn_pct
FROM ola_driver
GROUP BY City
ORDER BY churn_pct DESC;
```

Latest record per driver (window function)

```
SELECT *
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY Driver_ID ORDER BY reporting_month DESC) AS rn
    FROM ola_driver
) t
WHERE rn = 1;
```

Rating change month over month (LAG)

```
SELECT Driver_ID, reporting_month, Quarterly_Rating,
       Quarterly_Rating - LAG(Quarterly_Rating)
           OVER (PARTITION BY Driver_ID ORDER BY reporting_month) AS rating_change
FROM ola_driver;
```

Average business value: churned vs retained drivers

```
WITH d AS (
    SELECT Driver_ID,
           MAX(CASE WHEN LastWorkingDate IS NOT NULL THEN 1 ELSE 0 END) AS churned,
           AVG(Total_Business_Value) AS avg_bv,
           AVG(Quarterly_Rating) AS avg_rating,
           COUNT(*) AS active_months
    FROM ola_driver
    GROUP BY Driver_ID
)
SELECT churned, COUNT(*) AS drivers, AVG(avg_bv) AS avg_business_value,
       AVG(avg_rating) AS avg_rating, AVG(active_months) AS avg_active_months
FROM d
GROUP BY churned;
```

Monthly churn trend

```
SELECT reporting_month,
       COUNT(*) AS active_drivers,
       SUM(CASE WHEN LastWorkingDate IS NOT NULL THEN 1 ELSE 0 END) AS churned,
       ROUND(100.0 * SUM(CASE WHEN LastWorkingDate IS NOT NULL THEN 1 ELSE 0 END)
             / COUNT(*), 2) AS churn_pct
FROM ola_driver
GROUP BY reporting_month
ORDER BY reporting_month;
```

KPIs to put on a retention dashboard

KPI	Definition
Monthly churn rate	Drivers who left this month / drivers active this month
Retention by joining cohort	Share of drivers from each joining month still active after 3, 6, 12 months
Churn by segment	By city, rating band, tenure band, grade, income band
High-risk driver count	Drivers with churn probability above the chosen threshold
Average business value	Churned vs retained drivers
Intervention success rate	Retention of treated vs control drivers (A/B test)

16. Interview Questions and Answers

Project and business

Q1. Walk me through your project.

Use the 2-minute story in Section 1: problem, data, analysis, model, impact.

Q2. Why did you choose this project?

Driver retention directly affects revenue and rider experience. The project combines data cleaning, EDA, statistics, ML and business recommendations, the full analytics workflow.

Q3. Who would use your output?

Driver operations and retention teams: a ranked list of at-risk drivers with reasons, plus city- and segment-level dashboards for managers.

Q4. What are your top three recommendations?

Coaching for low-rated drivers, stronger onboarding and early incentives for new drivers, and city-specific retention plans for high-churn cities.

Q5. How would you measure whether the recommendations worked?

Run an A/B test: randomly split high-risk drivers into treatment and control groups, compare churn rates after 1-3 months with a two-proportion z-test, and compare the cost of the offer with the value of the drivers retained.

Data and statistics

Q6. How did you define churn?

Churn = 1 when LastWorkingDate is not null, which marks a driver's final month. Explain the row-level (8.46%) vs driver-level difference.

Q7. How did you handle missing values?

LastWorkingDate nulls are meaningful (not churned) and became the target. Age and Gender nulls were imputed with the median and mode; median because it is robust to outliers.

Q8. What is a p-value?

The probability of seeing a result at least as extreme as the observed one if the null hypothesis were true. Below the significance level (usually 0.05), reject the null.

Q9. Chi-square vs t-test?

Chi-square tests whether two categorical variables are related (Education vs Churn). A t-test compares the means of a numeric variable between two groups (Tenure for churners vs non-churners).

Q10. Why Welch's t-test?

The two groups have unequal sizes and variances; Welch's version does not assume equal variances.

Q11. Education was significant. Would you act on it?

Not much. The effect is small (about 1.5 percentage points), and with 19,000 rows small effects become significant. Statistical significance is not practical significance.

Q12. Type I vs Type II error?

Type I = false positive (rejecting a true null; in the model, flagging a driver who would stay). Type II = false negative (missing a real effect; missing a churner).

Q13. Correlation vs causation?

Low ratings are associated with churn, but the data doesn't prove that low ratings cause churn; both could stem from a third factor such as a bad area or low earnings. An experiment (A/B test) is needed to establish cause.

Modeling and evaluation

Q14. Why not use accuracy?

The accuracy paradox: predicting "no churn" for everyone gives 91.5% accuracy and catches nobody. Logistic Regression had 92% accuracy and 0.9% recall.

Q15. How did you handle class imbalance?

scale_pos_weight / class weights, stratified split, F1 and recall as metrics, and threshold tuning to 0.10. SMOTE and undersampling are alternatives.

Q16. Why threshold 0.10?

It maximized F1 and gave 76% recall. Missing a churner costs more than a false alarm, so recall matters more than precision.

Q17. Precision vs recall: which matters more here?

Recall, because a lost driver is expensive. But if the retention offer were very costly (e.g. a large cash bonus), precision would matter more and I would raise the threshold.

Q18. What does AUC = 0.895 mean?

If you pick one random churner and one random non-churner, the model gives the churner a higher score 89.5% of the time.

Q19. Why XGBoost / tree models?

Tabular data with non-linear relationships and interactions; trees need no scaling, handle outliers well, and give feature importance. Gradient boosting is usually the strongest approach on tabular data.

Q20. What is stacking and why use it?

Several different base models (XGBoost, LightGBM, CatBoost, Random Forest) feed their predictions into a meta-model (Logistic Regression). Different models make different errors, so the combination is more robust. It improved AUC to 0.895.

Q21. Bagging vs boosting?

Bagging trains models in parallel on bootstrap samples and averages them (reduces variance; Random Forest). Boosting trains models sequentially, each correcting the previous one's errors (reduces bias; XGBoost).

Q22. What is overfitting and how did you prevent it?

When a model memorizes training data and does badly on new data. Prevented with a held-out test set, cross-validation, limited tree depth, subsampling and regularization (gamma).

Q23. What is cross-validation?

Split training data into k folds; train on k-1 folds and validate on the remaining one, k times; average the scores. It gives a more reliable estimate than a single split.

Q24. Grid search vs random search vs Optuna?

Grid search tries every combination (slow). Random search samples combinations (faster, often as good). Optuna uses Bayesian optimization to learn from previous trials and focus on promising regions.

Q25. What is SHAP?

A method based on Shapley values that splits each prediction into feature contributions. It gives global importance and per-driver explanations. Top factors: rating, tenure/active months, city, business value.

Q26. Why one-hot encoding and not label encoding for City?

City codes have no natural order; label encoding would imply C29 > C1. One-hot creates independent 0/1 columns.

Q27. Why StandardScaler?

It puts features on the same scale (mean 0, standard deviation 1). Logistic Regression and distance-based models need it; tree models don't, but it keeps the pipeline consistent.

Reflection

Q28. What was the hardest part?

The feature mismatch bug (43 vs 44 features). Unnamed: 0 wasn't dropped and the preprocessor wasn't being saved, so the dashboard loaded a stale one. Lesson: always save the preprocessor with the model and regenerate artifacts when the code changes.

Q29. What would you do differently?

Prevent leakage with a driver- or time-based split and past-only features, fix date parsing, reframe the target as churn within the next N months, and validate the business impact with an A/B test.

Q30. How would you deploy and maintain this?

Score drivers monthly with a scheduled job, serve predictions through an API or dashboard, monitor data drift and model performance, and retrain when performance drops.

17. Cheat Sheet: Numbers and Facts to Remember

Item	Value
Rows x columns	19,104 x 14 (one row = one driver-month, 2019-2020)
Cities	29 (C1 to C29)
Row-level churn rate	8.46%
City churn range	5.67% (C29) to 12.5% (C17)
Chi-square, Education vs Churn	Chi-square = 9.51, df = 2, p = 0.0086 (significant, small effect)
Welch t-test, Tenure vs Churn	t = -23.13, p close to 0 (churners have much shorter tenure)
Train / test	15,283 / 3,821 rows (80/20 stratified); 323 churners in test
Features after encoding	43 (final pipeline); 48 in the notebook version
Always-"no churn" baseline accuracy	91.5%
Baseline XGBoost (weighted)	Recall 0.68, F1 0.39, AUC 0.86
Tuned XGBoost (no weighting)	Recall 0.20, F1 0.29, AUC 0.88
Final stacking ensemble, threshold 0.10	AUC 0.895, Recall 0.762, Precision 0.346, F1 0.476, Accuracy 0.858
Final confusion matrix	TN 3,033 | FP 465 | FN 77 | TP 246
Efficiency vs random targeting	About 4x
Top churn drivers	Quarterly Rating, Tenure / Active Months, City, Business Value, Income
Tools	Pandas, NumPy, Matplotlib, Seaborn, SciPy, scikit-learn, XGBoost, LightGBM, CatBoost, Optuna, SHAP, Streamlit

Glossary of core concepts

Term	Meaning
Supervised learning	Learning from labelled examples (here, rows with a known churn label)
Binary classification	Predicting one of two classes (churn / no churn)
Feature / target	Inputs the model uses (X) / the value it predicts (y)
Train / test split	Learn on one part of the data, measure on unseen data
Stratification	Keep class proportions the same across splits
Class imbalance	One class is much rarer than the other
Data leakage	Information unavailable at prediction time sneaks into training, inflating scores

Term	Meaning
Hyperparameter	A setting chosen before training (tree depth, learning rate)
Gradient boosting	Adding trees one by one, each fitting the remaining errors
Threshold	Probability cut-off that turns a score into a yes/no decision
Imputation	Filling missing values (median, mode)
One-hot encoding	Turning a category into separate 0/1 columns
Standardization	Rescaling to mean 0, standard deviation 1
Pickle	Python's way to save a trained object (model, preprocessor) to a file

Final tip for the interview

Lead with the business problem and the insight, not the algorithm. Quote 3-4 numbers confidently (8.46% churn, AUC 0.895, recall 76%, rating and tenure as top drivers). Explain why accuracy is misleading. Name one honest limitation and how you'd fix it. That combination of business sense, statistics, communication and self-awareness is what data analyst interviewers look for.
Good luck!